

02 · Motor de backtesting

Quant-Backtester · material de estudio

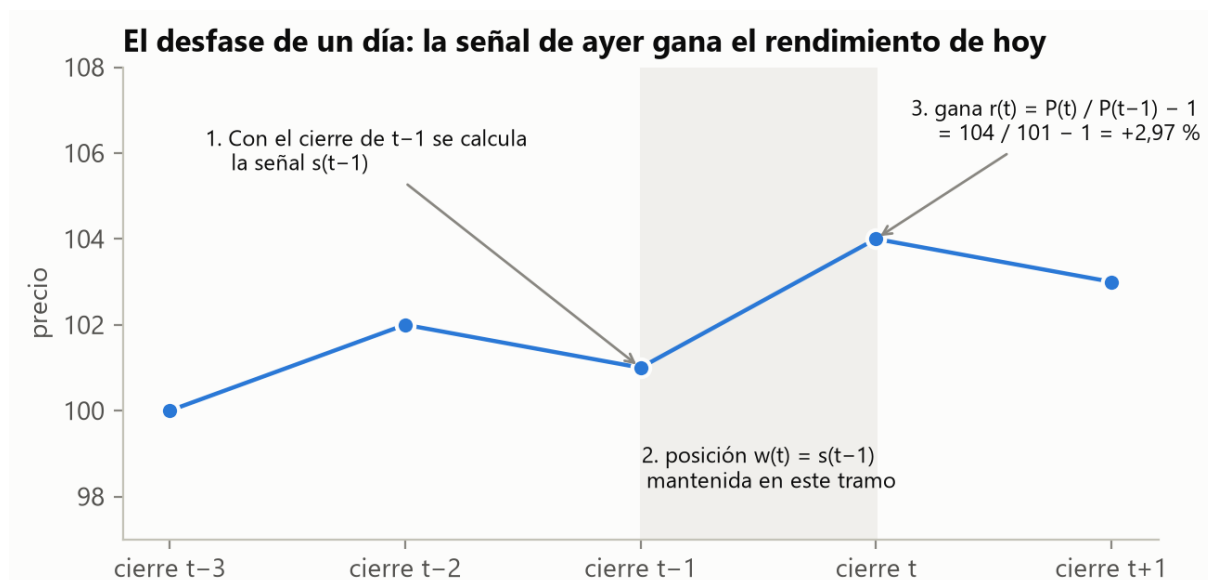
El motor convierte una tabla de señales en una serie de rendimientos netos y una curva de capital. Es poco código, seis funciones de una o dos líneas, y casi todo el riesgo está en los detalles: en qué día se aplica cada señal, qué cuenta como operar, cómo se agregan los activos y cómo se encadenan los días. Basta un desfase de una fila para convertir una estrategia mediocre en una máquina de imprimir dinero que no existe.

El capítulo cubre los TODO 2.1 a 2.6 de `src/backtest.py`: `signals_to_positions`, `compute_turnover`, `compute_costs`, `asset_pnl`, `to_portfolio` y `equity_curve`. Los orquestadores que los encadenan (`run_backtest`, `backtest_positions`) y el contenedor de resultados (`BacktestResult`) ya están hechos. El motor completo necesita además tu `simple_returns` del TODO 1.1. El checkpoint es `tests/test_backtest.py`, que incluye los dos tests que pide el README y un test de look-ahead, "el oráculo".

1. Intuición

Señal, posición, P&L. Una estrategia produce, para cada activo i y sesión t , una señal $s_{\{i,t\}}$ que vale 1 (largo), 0 (fuera) o -1 (corto). Es una opinión calculada con la información disponible al cierre de t , precios hasta P_t incluido. La posición $w_{\{i,t\}}$ es lo que de verdad tienes en cartera, y el P&L es la posición multiplicada por lo que se mueve el activo mientras la tienes. Señal y posición no son lo mismo: entre una y otra hay un desfase temporal, y ese desfase separa un backtest honesto de uno que ve el futuro. Por diseño, las estrategias nunca desplazan nada (lo dice el docstring de `src/strategies.py`): el desfase vive en un único sitio, `signals_to_positions`.

Quién cobra r_t . El rendimiento $r_t = P_t / P_{\{t-1\}} - 1$ mide lo que se mueve el activo desde el cierre de $t-1$ hasta el cierre de t . Solo lo cobra quien tenía la posición durante ese tramo, es decir, quien la había abierto como muy tarde en el cierre de $t-1$. Por tanto, la posición que cobra r_t , que llamamos w_t , solo puede depender de información disponible en el cierre de $t-1$. La señal más reciente que cumple esa condición es $s_{\{t-1\}}$, y por eso $w_t = s_{\{t-1\}}$.



Línea temporal de una sesión: la señal se calcula con el cierre de $t-1$, la posición w_t se mantiene hasta el cierre de t y cobra r_t

Momento	Qué sabes	Qué haces (lag = 1)
Cierre de $t-1$	Precios hasta $P_{\{t-1\}}$: calculas $s_{\{t-1\}}$	Ajustas la cartera a $w_t = s_{\{t-1\}}$ con una orden al cierre
Sesión t	Nada que puedas usar ya para w_t	Mantienes w_t
Cierre de t	P_t , y con él r_t : calculas s_t	Cobras $w_t \cdot r_t$ y ajustas la cartera a $w_{\{t+1\}} = s_t$

Por qué lag = 0 es trampa. Con lag = 0, $w_t = s_t$: la posición que cobra el movimiento entre el cierre de $t-1$ y el de t se decide con el cierre de t , es decir, cuando ese movimiento ya ha ocurrido. El caso extremo es el oráculo: $s_t = \text{signo de } r_t$, una señal perfectamente legítima porque al cierre de t conoces r_t . Sin desfase, $w_t \cdot r_t = |r_t|$ todos los días y nunca pierdes. En los datos sintéticos de los tests (3 activos, 5 años de paseo aleatorio, sin costes), esa estrategia multiplica el capital por unas 275 000. Con lag = 1 la misma señal se convierte en una apuesta a que mañana se repite el signo de hoy; en un paseo aleatorio eso no da ventaja y la equity termina en torno a 0,93. Por eso `signals_to_positions` lanza `ValueError` si lag < 1.

El look-ahead de lag = 0 es el error número uno de los backtests caseros, pero no el único. Todas estas variantes cuelan información del futuro: rellenar precios hacia atrás (`bfill`); normalizar una serie con la media o la desviación de toda la muestra; usar ventanas móviles centradas (`rolling(..., center=True)`); elegir parámetros con todo el histórico (lo verás en el walk-forward); elegir el universo sabiendo qué empresas sobrevivieron. La regla es la misma en todos los casos: la posición que cobra r_t solo puede usar datos hasta el cierre de $t-1$.

"Operar al cierre de mañana" sería lag = 2

Es tentador resumir `shift(1)` como "se opera al cierre de mañana con la información de hoy" (una versión anterior del README lo decía así), pero el código hace otra cosa: con $w_t = s_{t-1}$ y $r_t = P_t / P_{t-1} - 1$, la señal calculada con el cierre de $t-1$ se mantiene desde el cierre de $t-1$ hasta el de t . Es decir, el motor supone que **operas en el mismo cierre con el que calculas la señal**. Lo que sí es cierto es que la *posición* de mañana (w_t) se decide con la información de hoy (el cierre de $t-1$). Si operaras literalmente "al cierre de mañana", la posición abierta en el cierre de t solo cobraría a partir de r_{t+1} : eso es $w_{t+1} = s_{t-1}$, o sea, lag = 2.

Operar en el mismo cierre con el que calculas la señal es una aproximación habitual en backtests diarios. En la práctica calculas la señal unos minutos antes del cierre, con un precio casi definitivo, y envías una orden al cierre (*market-on-close*). La diferencia entre el precio que usaste y el cierre real es pequeña para estrategias lentas (un cruce de medias de 50 y 200 sesiones casi nunca cambia en diez minutos) y mayor para estrategias rápidas que reaccionan a movimientos del propio día; el slippage absorbe una parte. Con datos solo de cierre no puedes modelar la alternativa intermedia, operar en la apertura de t (el README lo reconoce en Limitaciones).

lag = 2 es la versión conservadora: calculas la señal con el cierre de $t-2$, tienes toda la sesión $t-1$ para preparar la orden y la ejecutas en el cierre de $t-1$.

Momento	lag = 1	lag = 2
Cierre de $t-2$		Calculas s_{t-2}
Sesión $t-1$		Preparas la orden con calma
Cierre de $t-1$	Calculas s_{t-1} y operas en ese mismo cierre	Ejecutas: $w_t = s_{t-2}$
Cierre de t	Cobras r_t	Cobras r_t

Es una prueba de robustez barata: si los resultados de una estrategia se hunden al pasar de lag = 1 a lag = 2, su ventaja depende de ejecutar exactamente en el cierre de la señal, y eso es frágil.

Operar cuesta. Cada vez que la posición cambia, compras o vendes. El turnover mide cuánto: $|w_t - w_{t-1}|$, en unidades de posición. Pasar de 0 a 1 es comprar la posición entera (turnover 1); mantener 1 no cuesta nada (turnover 0); pasar de +1 a -1 es vender el largo y abrir el corto (turnover 2); pasar de 1 a 0,5 es vender media posición (turnover 0,5). Se parte de estar fuera, $w_{-1} = 0$, así que tener posición el primer día significa haber comprado.

Los costes se expresan en puntos básicos: 1 bp = 0,01 % = 0,0001. La comisión es lo que cobra el intermediario; el slippage, la diferencia entre el precio que supone el backtest (el cierre) y el que consigues de verdad (medio diferencial entre compra y venta, impacto de tu orden). El modelo es lineal: cada unidad de turnover cuesta comisión más slippage. Con los valores por defecto, 5 + 5 = 10 bps = 0,001. Con 100 000 € en un solo activo, entrar cuesta 100 € y darle la vuelta a la posición, 200 €. Una estrategia que rota su cartera 20 veces al año paga del orden de un 2 % anual solo en costes.

La cartera. Cada activo tiene una ranura fija de $1/N$ del capital, y la posición escala esa ranura: con $w = 1$ la ranura está invertida en largo, con $w = 0$ está en liquidez (que rinde 0, porque `RISK_FREE` es 0) y con $w = -1$ está en corto. El rendimiento de la cartera es la suma de lo que aporta cada ranura dividida entre N , **siempre N**, aunque ese día haya activos fuera o sin dato. Si solo estás invertido en 2 de 10 activos, el 80 % del capital está en

liquidez; el motor no concentra el dinero en los activos con señal.

Capitalizar. El capital de hoy es el de ayer multiplicado por $1 +$ el rendimiento neto de hoy. La curva de equity es, por tanto, un producto acumulado. Sumar rendimientos sería suponer que cada día inviertes la misma cantidad fija, sin reinvertir lo ganado ni reponer lo perdido.

2. Formalización

Notación (la de los docstrings): $s_{i,t}$ señal, $w_{i,t}$ posición, $r_{i,t}$ rendimiento simple, N número de activos (columnas), b_{com} y b_{slip} comisión y slippage en bps, E_0 capital inicial (`initial`).

Posición:

$$w_{i,t} = s_{i,t-lag}$$

con $lag \geq 1$ y $w_{i,t} = 0$ en las primeras lag sesiones, en las que no existe señal previa.

Rendimiento del activo (TODO 1.1):

$$r_{i,t} = \frac{p_{i,t}}{p_{i,t-1}} - 1$$

Turnover y costes, por activo:

$$turnover_{i,t} = |w_{i,t} - w_{i,t-1}|$$

$$w_{i,-1} = 0$$

$$coste_{i,t} = turnover_{i,t} \cdot \frac{b_{com} + b_{slip}}{10000}$$

El coste del cambio de posición que se hace en el cierre de $t-1$ se apunta el día t , junto con el rendimiento r_t que esa nueva posición va a cobrar: es el precio de construir la posición del periodo t .

P&L bruto por activo, con los rendimientos ausentes contados como 0:

$$pnl_{i,t} = w_{i,t} \cdot r_{i,t}$$

Agregación a la cartera y rendimiento neto:

$$bruto_t = \frac{1}{N} \sum_{i=1}^N pnl_{i,t}$$

$$neto_t = bruto_t - \frac{1}{N} \sum_{i=1}^N coste_{i,t}$$

Curva de capital:

$$equity_t = E_0 \cdot \prod_{s \leq t} (1 + neto_s)$$

El primer valor de la curva ya incluye el primer rendimiento: es $E_0 \cdot (1 + neto_0)$, no E_0 .

Ejemplo completo a mano. Un activo ($N = 1$), $lag = 1$ y costes de 10 bps:

t	s_t	w_t	r_t	turnover_t	coste_t
0	1	0	NaN	0	0

t	s_t	w_t	r_t	turnover_t	coste_t
1	1	1	2 %	1	0,001
2	-1	1	-1 %	0	0
3	-1	-1	3 %	2	0,002
4	0	-1	-2 %	0	0
5	0	0	1 %	1	0,001

t	pnl_t	neto_t	equity_t
0	0	0	1,0000
1	0,020	0,019	1,0190
2	-0,010	-0,010	1,0088
3	-0,030	-0,032	0,9765
4	0,020	0,020	0,9961
5	0	-0,001	0,9951

Léelo fila a fila. El día 0 la estrategia ya dice "largo", pero aún no hay posición: no se gana ni se paga nada. El día 2 la señal pasa a -1, pero la posición sigue en +1 y sufre la caída del 1 %; el cambio de largo a corto se hace en el cierre del día 2 y se apunta el día 3 (turnover 2, coste 0,002), justo cuando el activo sube un 3 % contra el corto. El día 5 se cierra la posición: no hay P&L, pero sí coste. Es el mismo patrón de posiciones y turnover que comprueba `test_run_backtest_flip_costs`.

Los dos tests del README. Señal constante en 1 sobre un solo activo: $w = [0; 1; 1; \dots]$ y $\text{turnover} = [0; 1; 0; \dots]$. Entonces $\text{neto}_0 = 0$, $\text{neto}_1 = r_1 - c$ con $c = 0,001$, y $\text{neto}_t = r_t$ desde el día 2. Multiplicando, y usando que los factores $1 + r_t$ encadenados dan P_T / P_0 :

$$\text{equity}_T = \frac{P_T}{P_0} \cdot \frac{1 + r_1 - c}{1 + r_1}$$

Es el buy-and-hold menos un coste inicial. Fíjate en dos detalles. La estrategia cobra todo el recorrido desde P_0 , porque compra en el cierre del día 0 con la señal del día 0. Y el coste se resta del rendimiento del día 1, no del capital, por eso el factor es $(1 + r_1 - c)/(1 + r_1)$ y no exactamente $1 - c$; la diferencia es de segundo orden.

`test_readme_constant_signal_one_is_buy_and_hold_minus_initial_cost` comprueba las cuatro cosas: $\text{neto}_0 = 0$ exacto, $\text{coste}_1 = 0,001$, rendimientos idénticos a los del activo desde el día 2 y la equity final de la fórmula.

Señal constante en 0: posiciones, turnover, costes y rendimientos son 0 todos los días y la equity vale exactamente 1. `test_readme_constant_signal_zero_gives_zero_return` compara con `==`, sin tolerancia: no puede colarse ni un NaN ni ruido numérico.

El oráculo. Con $\text{lag} = 0$ y $s_t = \text{signo de } r_t$:

$$w_t \cdot r_t = \text{sign}(r_t) \cdot r_t = |r_t|$$

Con una volatilidad anual del 20 %, la diaria es de un 1,26 % y el valor medio de $|r_t|$ es de aproximadamente un 1 %. Ganar un 1 % diario durante 1260 sesiones da 1,01 elevado a $1260 \approx 2,8 \times 10^5$. `test_lookahead_oracle_cannot_win` exige que la equity final sea menor que 3 con el motor completo; si falta el desfase, falla con el mensaje "Demasiado bueno para ser verdad: ¿falta el shift?".

Dos simplificaciones del modelo

Primera: pesos constantes $w_{\{i,t\}}/N$ cada día equivalen a reequilibrar en cada cierre para volver a $1/N$ por activo, y ese reequilibrio no paga costes, porque el turnover solo mide cambios de w . Con un activo y $w = 1$ es exacto; con varios activos, el "buy-and-hold" del motor es en realidad una cartera equiponderada reequilibrada a diario. Segunda: si un activo tiene un hueco más largo que `max_fill`, el rendimiento del día en que reaparece también es NaN (falta el precio de ayer) y cuenta como 0, así que el movimiento ocurrido durante el hueco no se cobra ni se paga. Con ETF y acciones grandes casi no pasa; si aparece en tus datos, investigalo.

3. Del papel al código

Lo que ya está hecho. Conviene que leas `src/backtest.py` antes de empezar; el flujo es este:

```
signals --(2.1)--> positions --+--(2.4)--> P&L por activo: w_t * r_t
                                     +-+(2.2)--> turnover --(2.3)--> costes por activo
(2.5) P&L, costes y turnover por activo --> cartera (suma / N)
neto = bruto - costes --(2.6)--> equity
```

- `run_backtest(prices, signals, commission_bps, slippage_bps, lag, vol_target)` valida las entradas, llama a tu `signals_to_positions`, aplica opcionalmente el volatility targeting (TODO 6.1, extensión) y delega en `backtest_positions`.
- `backtest_positions(prices, positions, ...)` recibe posiciones **ya desplazadas** y no vuelve a desplazar. Calcula `simple_returns(prices)` (tu TODO 1.1), el turnover, los costes y el P&L por activo; agrega a cartera con `to_portfolio` tres veces (P&L, costes y turnover); resta costes al bruto y capitaliza con `equity_curve`. La usan también el walk-forward, que encadena posiciones de varias ventanas, y el volatility targeting.
- `_validate_inputs` exige DataFrames con `DatetimeIndex` ordenado y sin fechas repetidas (de ahí `clean_prices`), las mismas fechas y las mismas columnas en el mismo orden, y señales o posiciones sin NaN ni infinitos. Los precios sí pueden tener NaN. Si ves "positions contiene NaN o infinitos", el problema está en tu TODO 2.1.
- `BacktestResult` guarda, sobre el mismo índice de fechas, `returns` (netos), `gross_returns`, `costs`, `equity`, `turnover` (de cartera), `positions`, `asset_turnover` y `asset_returns`. La propiedad `cumulative_turnover` acumula el turnover, y `slice(start, end)` recorta un subperiodo y recalcula la equity desde 1 con tu `equity_curve`.

Las seis piezas trabajan sobre tablas completas, fechas × activos, sin bucles. Todo lo dicho en el capítulo 01 sobre alineación por etiquetas, `shift` y propagación de NaN sigue valiendo aquí.

TODO 2.1 · signals_to_positions

Qué hace. Convierte señales en posiciones desplazándolas `lag` sesiones hacia el futuro, empezando fuera del mercado, y rechaza los lag que permitirían ver el futuro.

Firma.

```
def signals_to_positions(signals: pd.DataFrame, lag: int = 1) -> pd.DataFrame:
```

Convenciones que comprueban los tests.

- Por defecto, `w_t = s_{t-1}`: `A = [1, 1, 0, -1, -1]` da `[0, 1, 1, 0, -1]` y `B = [0, 1, 1, 1, 0]` da `[0, 0, 1, 1, 1]` (`test_signals_to_positions_shifts_one_day`).
- Con `lag = 2`: `[1, -1, 1, 1]` da `[0, 0, 1, -1]` (`test_signals_to_positions_lag_two`).
- `lag = 0` lanza `ValueError` (`test_signals_to_positions_rejects_lag_zero`); el docstring lo extiende a cualquier `lag < 1`.
- Cambiar las señales a partir del día `k` no cambia ninguna posición hasta el día `k` incluido (`test_signals_to_positions_never_uses_future_signals`).
- Misma forma que la entrada y ningún NaN (`test_signals_to_positions_no_nan_same_shape`).

- En el motor completo, sin desfase no pasa `test_lookahead_oracle_cannot_win`.

Pseudocódigo.

1. Si lag es menor que 1, lanza ValueError con un mensaje que explique que sería look-ahead (la posición usaría el cierre que intenta predecir).
2. Desplaza toda la tabla de señales lag filas hacia fechas posteriores, sin tocar el índice ni las columnas.
3. Las primeras lag filas no tienen señal previa: ponlas a 0 (fuera del mercado).
4. Devuelve la tabla.

Pistas.

Pista 1

La posición de hoy es la señal de hace lag sesiones. Antes de que exista esa señal estás fuera. No hay nada más: no se suaviza, no se redondea, no se trata cada activo por separado.

Pista 2

`shift(n)` con $n > 0$ mueve los valores hacia fechas posteriores y conserva el índice; las n primeras filas quedan vacías. Repasa el ejemplo de juguete del capítulo 01: con [10, 11, 12], `shift(1)` da [NaN, 10, 11]. Funciona igual sobre un DataFrame entero, columna a columna.

Pista 3

`shift` admite `fill_value`, así resuelves el hueco inicial en el mismo paso y no queda ningún NaN que `_validate_inputs` rechace. Haz la comprobación de lag lo primero, antes de tocar los datos.

TODO 2.2 · compute_turnover

Qué hace. Mide, para cada activo y día, cuánto cambia la posición respecto a la sesión anterior, partiendo de estar fuera.

Firma.

```
def compute_turnover(positions: pd.DataFrame) -> pd.DataFrame:
```

Convenciones que comprueban los tests.

- Valores conocidos: A = [1; 1; -1; 0; 0,5] da [1; 0; 2; 1; 0,5] y B = [0, 0, 0, 1, 1] da [0, 0, 0, 1, 0] (`test_compute_turnover_known_values`).
- La primera fila es |w_0|: [1, 1, 1] da [1, 0, 0] (`test_compute_turnover_first_day_counts_as_entry`).
- Pasar de +1 a -1 cuenta 2 (`test_compute_turnover_flip_counts_two`).
- Ningún NaN (`test_compute_turnover_no_nan`).
- En el motor completo: turnover [0, 1, 0, 2, 0, 1] en `test_run_backtest_flip_costs` y turnover acumulado 3 en `test_backtest_result_cumulative_turnover`.

Pseudocódigo.

1. Construye "la posición de la sesión anterior" para cada activo; en la primera fila, la posición anterior es 0 (se parte de estar fuera).
2. Resta, celda a celda, la posición actual menos la anterior.
3. Toma el valor absoluto de cada celda.
4. Devuelve la tabla, con la misma forma y sin NaN.

Pistas.

Pista 1

El turnover es la distancia entre la posición de hoy y la de ayer; el sentido (compra o venta) da igual, ambos cuestan. Piensa qué vale "la posición de ayer" el primer día: no es la posición del primer día, es 0.

Pista 2

`diff()` calcula $x_t - x_{t-1}$ columna a columna; `abs()` quita el signo; `shift(1, fill_value=0)` construye "la posición de ayer" con $w_{-1} = 0$. Con dos de estas tres herramientas tienes suficiente.

Pista 3

`diff()` deja NaN en la primera fila, y rellenarlo con 0 es un error sutil: equivale a suponer que antes de empezar ya tenías la posición w_0 , así que la entrada sale gratis. El valor correcto de esa fila es $|w_0|$. En `run_backtest` con $\text{lag} \geq 1$, w_0 siempre es 0 y el error pasaría inadvertido; pero `backtest_positions` acepta posiciones de cualquier origen (el walk-forward, por ejemplo) y ahí sí importa. Si construyes la posición anterior con `fill_value=0`, el caso sale solo.

TODO 2.3 · compute_costs

Qué hace. Convierte el turnover de cada activo en coste, en tanto por uno del capital de su ranura.

Firma.

```
def compute_costs(
    turnover: pd.DataFrame,
    commission_bps: float = config.COMMISSION_BPS,
    slippage_bps: float = config.SLIPPAGE_BPS,
) -> pd.DataFrame:
```

Convenciones que comprueban los tests.

- Con los valores por defecto (5 + 5 bps): turnover A = [1, 2, 0] da [0,001; 0,002; 0] y B = [0,5; 0; 1] da [0,0005; 0; 0,001] (`test_compute_costs_default_is_10_bps_per_unit_of_turnover`).
- Con 2 + 3 bps, un turnover de 1 cuesta 0,0005 (`test_compute_costs_custom_bps`).
- Con 0 + 0 bps, el coste es exactamente 0 (`test_compute_costs_zero_bps`).
- En el motor completo, los costes nunca son negativos y el neto es bruto menos costes (`test_run_backtest_net_is_gross_minus_costs`).

Pseudocódigo.

1. Suma la comisión y el slippage, ambos en puntos básicos.
2. Pasa esa suma a tanto por uno (1 bp = 0,0001).
3. Multiplica cada celda de turnover por ese número.
4. Devuelve la tabla, con la misma forma que turnover.

Pistas.

Pista 1

Es un cambio de unidades seguido de una multiplicación. Comisión y slippage se suman porque se pagan los dos en cada operación; no se multiplican entre sí ni se elige uno.

Pista 2

Un DataFrame multiplicado por un escalar multiplica todas sus celdas y conserva índice y columnas. No necesitas bucles, `apply` ni nada específico de pandas.

Pista 3

Divide entre 10 000, no entre 100: dividir entre 100 es pasar de porcentaje a tanto por uno, y los bps son centésimas de porcentaje. Comprueba tu resultado contra el docstring: 5 + 5 bps con turnover 1 tiene que dar 0,001.

TODO 2.4 · asset_pnl

Qué hace. Calcula el P&L bruto de cada activo: posición de hoy por rendimiento de hoy, con los rendimientos ausentes contados como 0.

Firma.

```
def asset_pnl(positions: pd.DataFrame, asset_returns: pd.DataFrame) -> pd.DataFrame:
```

Convenciones que comprueban los tests.

- No desplace nada: posiciones [0, 1, 1, -1] con rendimientos [NaN; 0,02; -0,01; 0,03] dan [0; 0,02; -0,01; -0,03] (`test_asset_pnl_multiplies_without_shifting`).
- Un rendimiento NaN cuenta como 0 aunque la posición sea 1, y el resultado no tiene NaN (`test_asset_pnl_nan_returns_count_as_zero`).
- En el motor completo, el día 0 el neto es exactamente 0 (`test_readme_constant_signal_one_is_buy_and_hold_minus_initial_cost`).

Pseudocódigo.

1. Sustituye por 0 los rendimientos que faltan (el primer día, los huecos).
2. Multiplica, celda a celda, la posición de cada fecha por el rendimiento de esa misma fecha.
3. Devuelve la tabla.

Pistas.

Pista 1

El desfase ya lo puso `signals_to_positions`: en la fila `t` de `positions` está la posición que cobra `r_t`, que está en la fila `t` de `asset_returns`. Fila con fila, sin mover nada.

Pista 2

El producto de dos DataFrames con el mismo índice y las mismas columnas opera celda a celda, alineando por etiquetas. `fillna(0)` sustituye los NaN por ceros.

Pista 3

En coma flotante, $0 \times \text{NaN} = \text{NaN}$: que la posición sea 0 no te salva del NaN. Rellena con 0 los rendimientos antes de multiplicar, o el producto después; las dos opciones dan lo mismo. Y no desplaces nada aquí: un segundo `shift` convertiría tu lag 1 en lag 2 sin que te dieras cuenta.

TODO 2.5 · to_portfolio

Qué hace. Agrega una tabla por activo (P&L, costes o turnover) a una serie de cartera equiponderada: la suma de las columnas dividida entre el número total de activos.

Firma.

```
def to_portfolio(asset_values: pd.DataFrame) -> pd.Series:
```

Convenciones que comprueban los tests.

- Divide entre el número total de columnas, aunque algún activo esté a 0 o tenga NaN: con 4 activos, las filas [0,02; 0; 0,01; 0], [0; 0; 0; 0] y [NaN; 0,03; 0; 0] dan 0,03/4, 0 y 0,03/4 (`test_to_portfolio_divides_by_total_number_of_assets`).
- Devuelve una Series con las mismas fechas que la entrada (`test_to_portfolio_keeps_dates`).
- En el motor completo, una cartera de dos activos idénticos con señales idénticas rinde lo mismo que uno solo (`test_run_backtest_portfolio_of_identical_assets_equals_single_asset`).

Pseudocódigo.

1. Cuenta N, el número total de columnas (activos) de la tabla.
2. Para cada fecha, suma los valores de todos los activos, contando los NaN como 0.
3. Divide cada suma entre N.
4. Devuelve una serie con un valor por fecha y el mismo índice.

Ejemplo con 3 activos, para ver por qué la media no sirve:

Fecha	A	B	C	suma / N	media de los no NaN
d1	0,02	0,00	0,01	0,0100	0,0100
d2	NaN	0,03	0,00	0,0100	0,0150
d3	0,00	0,00	0,00	0,0000	0,0000

El día d2, el activo A no tiene dato, pero su tercio del capital sigue ahí y rinde 0. La media reparte ese tercio entre B y C solo ese día: es como apalancarlos de forma aleatoria cada vez que falta un precio.

Pistas.

Pista 1

Cada activo tiene una ranura fija de 1/N del capital. La cartera suma lo que aporta cada ranura. Una ranura sin dato o fuera del mercado aporta 0, pero sigue contando en N.

Pista 2

`sum(axis=1)` suma a lo ancho y devuelve una Series con un valor por fecha; `sum(axis=0)`, el valor por defecto, sumaría a lo largo de las fechas y te daría un valor por activo. El número de columnas está en `shape` o en `len(df.columns)`.

Pista 3

`sum` ignora los NaN por defecto, y una fila entera de NaN suma 0 (porque `min_count=0`), que es justo lo que pide el docstring. `mean(axis=1)` también ignora los NaN, pero divide entre el número de valores presentes, no entre N: con NaN, son cosas distintas.

TODO 2.6 · equity_curve

Qué hace. Capitaliza una serie de rendimientos diarios y devuelve la evolución del capital, partiendo de `initial`.

Firma.

```
def equity_curve(returns: pd.Series, initial: float = 1.0) -> pd.Series:
```

Convenciones que comprueban los tests.

- Valores conocidos: [0,10; -0,10; 0; 0,05] da [1,1; 0,99; 0,99; 1,0395] (`test_equity_curve_known_values`).
- El capital inicial multiplica toda la curva: con `initial=100`, [0,10; -0,10] da [110; 99] (`test_equity_curve_initial_capital`).
- Mismo índice que `returns`, sin filas añadidas (`test_equity_curve_same_index`).
- Tras `slice`, el primer valor de la equity es 1 + el primer rendimiento del tramo (`test_backtest_result_slice_rebases_equity`).
- En el motor completo, la equity final de los dos tests del README y del oráculo.

Pseudocódigo.

1. Convierte cada rendimiento en su factor de crecimiento: 1 + rendimiento.
2. Recorre las fechas en orden y calcula, para cada una, el producto de todos los factores hasta esa fecha (incluida).
3. Multiplica la serie resultante por el capital inicial.
4. Devuelve una serie con el mismo índice: el primer valor ya incluye el primer rendimiento.

Ejemplo: con rendimientos [0,05; -0,02; 0,01] e `initial = 100`, la curva es [105; 102,9; 103,929].

Pistas.

Pista 1

Cada día, el capital se multiplica por un factor. Lo que se encadena son los factores $1 + r$, no los rendimientos.

Pista 2

`cumprod()` devuelve el producto acumulado de una Series y conserva el índice: sobre [2, 3, 4] da [2, 6, 24].

Pista 3

Si aplicas el producto acumulado directamente a los rendimientos, [0,10; -0,10] da [0,10; -0,01], que no significa nada. No antepongas un 1 a la serie (cambiaría la longitud y el índice) y no uses `cumsum`: eso es interés simple.

4. Errores típicos

Sin desfase

Devolver las señales tal cual o aceptar $\text{lag} = 0$. Síntoma: equities espectaculares, Sharpe de dos cifras, una estrategia "perfecta". Lo detectan `test_signals_to_positions_shifts_one_day`, `test_signals_to_positions_rejects_lag_zero` y `test_lookahead_oracle_cannot_win`, que falla con "Demasiado bueno para ser verdad: ¿falta el shift?".

Desfase en el sentido equivocado

Usar un `shift` negativo. Síntoma: la posición de hoy es la señal de mañana; el resultado es igual de espectacular que sin desfase. Lo detecta `test_signals_to_positions_never_uses_future_signals`: alterar las señales a partir del día k cambia posiciones anteriores a k .

NaN en las primeras filas

Desplazar sin rellenar. Síntoma: `ValueError: positions contiene NaN o infinitos` al ejecutar `run_backtest`. Lo detecta `test_signals_to_positions_no_nan_same_shape`.

Desplazar dos veces

Hacer `shift` también en `asset_pnl` (o en una estrategia). Síntoma: el motor funciona con un lag efectivo de 2; el día 1 no se cobra el rendimiento y la equity del test de señal constante en 1 no cuadra. Lo detectan `test_asset_pnl_multiplies_without_shifting` y `test_readme_constant_signal_one_is_buy_and_hold_minus_initial_cost`.

Entrada gratis

Calcular el turnover con `diff()` y rellenar la primera fila con 0. Síntoma: una posición inicial distinta de 0 no paga coste. Lo detectan `test_compute_turnover_first_day_counts_as_entry` y `test_compute_turnover_known_values`; el motor completo no, porque con $\text{lag} \geq 1$ la primera posición siempre es 0.

Turnover con signo

Olvidar el valor absoluto. Síntoma: las ventas generan costes negativos y operar "da dinero". Lo detectan `test_compute_turnover_known_values` y `test_run_backtest_net_is_gross_minus_costs`, que exige costes no negativos.

Unidades de los costes

Dividir entre 100 o entre 1000 en vez de entre 10 000, multiplicar comisión por slippage o usar solo uno de los dos. Síntoma: costes 100 veces mayores, nulos o que ignoran un parámetro. Lo detectan `test_compute_costs_default_is_10_bps_per_unit_of_turnover` y `test_compute_costs_custom_bps`.

NaN en el P&L

No rellenar los rendimientos ausentes. Síntoma: NaN el primer día y en cada hueco, aunque la posición sea 0. Lo detecta `test_asset_pnl_nan_returns_count_as_zero`.

Media en vez de suma entre N

Usar `mean(axis=1)` o dividir entre el número de activos invertidos. Síntoma: los días con NaN o con activos fuera, el peso de los demás se infla. Lo detecta `test_to_portfolio_divides_by_total_number_of_assets`.

Sumar en el eje equivocado

Usar `sum()` sin `axis=1`. Síntoma: un valor por activo en vez de uno por fecha. Lo detectan `test_to_portfolio_divides_by_total_number_of_assets` (que exige una Series de 3 valores) y `test_to_portfolio_keeps_dates`.

Equity con sumas o con un valor de más

Usar `cumsum`, multiplicar los rendimientos en lugar de los factores, anteponer un 1 u olvidar `initial`. Lo detectan `test_equity_curve_known_values`, `test_equity_curve_same_index` y `test_equity_curve_initial_capital`.

5. Preguntas y ejercicios

1. Con señales [0, 1, 1, 1, -1, 0], calcula a mano las posiciones y el turnover con lag = 1 y con lag = 2. ¿Cuántos bps pagas en total con 5 + 5 bps?
2. Explica con la línea temporal por qué lag = 0 no se puede ejecutar en la realidad con datos de cierre, aunque la señal use solo información "de hoy".
3. Una estrategia tiene un CAGR del 25 % con lag = 1 y del 3 % con lag = 2. ¿Qué te dice eso de la estrategia? ¿La pondrías a operar?
4. Una estrategia tiene un turnover anual de 25. ¿Cuánto le cuestan al año los costes con 10 bps? ¿Y con 30 bps?
5. En una cartera de 10 activos, un día solo AAPL pasa de +1 a -1. ¿Cuánto vale el turnover de cartera ese día? ¿Y el coste, en bps del capital total?
6. En la tabla de `to_portfolio`, ¿qué peso efectivo tienen B y C el día d2 si usas la media? ¿Por qué eso contradice el diseño "una ranura de 1/N por activo"?
7. Con rendimientos +50 % y -50 %, ¿qué dan la suma acumulada y el producto acumulado de los factores? ¿Cuál describe tu dinero?
8. En el test de señal constante en 1, ¿por qué la equity final es $(P_T / P_0) \cdot (1 + r_1 - c) / (1 + r_1)$ y no $(P_T / P_0) \cdot (1 - c)$? ¿Cuál de las dos es más realista?
9. ¿Por qué el coste del cambio de posición hecho en el cierre de $t-1$ se apunta el día t y no el día $t-1$?
10. Ejercicio (notebook `02_estrategias.ipynb`). Ejecuta `run_backtest` para cada estrategia con `lag=1` y con `lag=2` y compara la equity final y el Sharpe. ¿Qué estrategia es más sensible al retraso? ¿Encaja con lo rápida que es su señal?
11. Ejercicio (notebook). Construye el oráculo con los precios reales (señal = signo del rendimiento de hoy, 0 el primer día). Pásalo por `run_backtest` sin costes y, después, directamente a `backtest_positions` como si ya fueran posiciones, que es exactamente lag = 0. Compara las dos equities finales y explica por qué `backtest_positions` exige posiciones ya desplazadas.

6. Checkpoint

```
python -m pytest tests/test_backtest.py
python -m pytest -k signals_to_positions
python -m pytest -k compute_turnover
python -m pytest -k compute_costs
python -m pytest -k asset_pnl
python -m pytest -k to_portfolio
python -m pytest -k equity_curve
python -m pytest -k "readme or oracle"
python -m pytest tests/test_data.py tests/test_backtest.py
```

El primero es el checkpoint del capítulo. Los comandos con `-k` seleccionan los tests cuyo nombre contiene el texto, para ir TODO a TODO. Usa los nombres completos de las funciones: `-k turnover` también seleccionaría `test_annual_turnover_known_value` (TODO 4.9, en `tests/test_metrics.py`) y `-k costs` un test del walk-forward. `-k "readme or oracle"` ejecuta los dos tests del README y el del oráculo.

Antes de empezar verás `27 skipped` en `tests/test_backtest.py`: el autocorrector convierte cada `NotImplementedError` en un skip y, al final de la salida, lista los TODO pendientes con cuántos tests espera cada uno. Los ocho tests del motor completo (`test_readme_...`, `test_lookahead_oracle_cannot_win`, `test_run_backtest_...` y `test_backtest_result_...`) siguen saliendo como skipped mientras quede cualquiera de los TODO 1.1 y 2.1 a 2.6; el motivo del skip (visible con `-rs`) indica el primero que encuentran. Un TODO implementado pero incorrecto sale como **FAILED** con el mensaje del `assert`. Recuerda que un `FutureWarning` o `DeprecationWarning` emitido desde `src/` también hace fallar el test.

Resultado esperado al terminar: `27 passed` en `tests/test_backtest.py`, y `44 passed` con el último comando, que ejecuta los capítulos 01 y 02 juntos. Con esto ya puedes llamar a `run_backtest` con `buy_and_hold`, que viene hecha; el notebook `02_estrategias.ipynb` completo necesita además las estrategias (capítulo 03) y las métricas (capítulo 04).